

Smart Parking System

Security Guard Module — MVC Architecture Documentation

ACADEMIC DOCUMENTATION

Executive Summary / විධායක සංක්ෂිප්තය

This document outlines the software architecture implementation for the **Security Guard Module** of the Smart Parking System. Developed strictly following the **Model-View-Controller (MVC)** design pattern, the system separates user presentation, request routing, and core domain business logic.

1. High-Level MVC Architecture Diagram



2. Detailed Layer Specification

LAYER	COMPONENT / FILE PATH	RESPONSIBILITY
VIEW	<code>security-guard.html</code> <code>security-gate.js</code>	- Renders dashboard UI for ESP32 status and barrier control. - Asynchronous HTTP requests (`fetch`) for barrier override. - Handles live vehicle animation loop and dark/light themes.
CONTROLLER	<code>SecurityGuardController.java</code> <code>BarrierController.java</code>	- SecurityGuardController: Maps `GET /security-guard` to HTML view. - BarrierController: Handles REST endpoints (`/api/gates/status`, `/api/gates/entry/open`, `/api/gates/exit/close`).
MODEL	<code>BarrierService.java</code> <code>GateStatus / GateAction /</code> <code>GateType</code> <code>GateControlRequest /</code> <code>GateStatusResponse</code>	- Maintains in-memory state for Entrance and Exit barriers. - Handles IoT communication with ESP32 microcontrollers. - Enforces domain rules via Enums and structured DTO payloads.

3. End-to-End Execution Flow Example (Barrier Open/Close)

1

User Action (View): The Security Guard clicks "Open Barrier" on the web dashboard. `security-gate.js` catches the event and sends an HTTP POST request to `/api/gates/entry/open`.

2

Request Interception (Controller): `BarrierController.java` receives the request and delegates execution to `BarrierService.java`.

3

State & Business Processing (Model): `BarrierService` communicates with the ESP32 controller, updates internal state to `GateStatus.OPEN`, and builds a `GateStatusResponse` DTO.

4

UI Synchronization (View Update): `security-gate.js` receives the JSON response, animates the barrier arm upward (-52°), sets signal light to GREEN, and logs the activity.

4. Verification & Code Quality

The implementation was validated using automated Maven unit and integration tests. All **15 unit tests** across controller and service layers passed cleanly with zero failures.

✓ BUILD SUCCESS — 15 Tests Executed, 0 Failures, 0 Errors.